

Multiplayer Prší — Backend Roadmap

junior → mid

*Cíl: postavit server-authoritative multiplayer backend pro Prší a přitom si do hloubky osahat klíčové koncepty a patterns. Napsat to **sám** (paměť + kód + úsudek).*

Klient = Unity (jen view + input). Pravdu o hře drží server.

Stack je koncept-first: u každé fáze je primárně Rails a vedle .NET ekvivalent — roadmapa slouží, ať zvolíš Rails nebo C#/ASP.NET Core. Patterns jsou jazykově nezávislé; mění se jen nástroje.

Jak roadmapu číst

Každá fáze má: **Koncept** (co a proč) · **Postav** (co přidat do Prší) · **Hloubka** (kam jít, aby to bylo mid) · **Rails** / **.NET** (nástroje) · **Reference** · **Ověření** (otázka — když na ni umíš odpovědět z hlavy, fázi máš).

Časový odhad: ~6–8 týdnů při 3–4 h/den (odhad, ne slib).

Fáze

Fáze 0 — Setup — 1–2 dny

Koncept. Kostra projektu. **Postav.** Rails 8 API-only + Postgres + RSpec + Rubocop + Git (nebo dotnet new webapi + EF Core + xUnit). **Hloubka.** Co API-only režim vypne (views, asset pipeline) a proč to nechceš. **Reference.** https://guides.rubyonrails.org/api_app.html · <https://learn.microsoft.com/aspnet/core/web-api/> **Ověření.** Proč API-only? Co tím ztrácíš a proč ti to nevadí?

Fáze 1 — Doménový port + TDD — cca týden

Koncept. Entity vs Value Object, agregát, business pravidla. Čistá doména bez DB a frameworku. **Postav.** Přepiš pravidla karet (máš odladěná v Prsi.Core): Card jako value object (immutable suit+rank), can_play_on?, efekty sedmy/dámy/esa. Žádný ActiveRecord. TDD. **Hloubka.** Proč je karta value object (rovná se hodnotou) a hra entita (má identitu). Immutabilita. **Trap:** nepiš C# v Ruby (žádný factory/interface). V C# naopak reuse Prsi.Core přímo. **Reference.** <https://martinfowler.com/bliki/ValueObject.html> · <https://martinfowler.com/eaCatalog/domainModel.html> **Ověření.** Proč je Card value object a Game entita? Kde je ta hranice?

Fáze 2 — Persistence — cca týden — nejtěžší nový kus

Koncept. Mapování domény do DB. Agregát → tabulky. Zdroj pravdy v Postgresu. **Postav.** Tabulky users, games, players, moves, stav rukou. Rozhodni: (A) snapshot stavu, nebo (B) seed + log Moves a „přehraj“ (event sourcing; seedovaný deck to umožňuje). Doporučení: ukládej Moves i snapshot = CQRS read model. **Hloubka.** AR asociace, indexy, transakce, N+1 (includes/joins/preload), perzistence agregátu bez nekonzistentního stavu. **Reference.** https://guides.rubyonrails.org/active_record_basics.html · https://guides.rubyonrails.org/active_record_querying.html · <https://learn.microsoft.com/ef/core/> **Ověření.** Kde je zdroj pravdy? Když server spadne uprostřed tahu, jak se hra obnoví?

Fáze 3 — State machine — 3–5 dní

Koncept. Finite state machine, přechody, guards. Lifecycle hry. **Postav.** `waiting_for_players` → `playing` → `finished`. Zakázat `finished` → `playing` (musí vyhodit chybu). První testy. **Hloubka.** Guards (do `playing` jen při ≥ 2 hráčích), callbacky na přechodu, proč stav nepatří do controlleru. **Rails / .NET.** Gem **AASM** / knihovna **Stateless** nebo vlastní enum + guardy. **Reference.** <https://github.com/aasm/aasm> · <https://rubyguides.dev/guides/ruby-state-machine-gem/> **Ověření.** Napiš test, že `finished` → `playing` vyhodí chybu a nezmění stav. Proč guard, ne `if` v controlleru?

Fáze 4 — Service objekty — 3–5 dní

Koncept. Aplikační vrstva. Orchestrace mimo model i controller. **Postav.** `PlayCardService.call(...)`, `JoinGameService`. Vytáhni změt z Unity `CardManager.PlayCard` (validace + efekt + výhra + posun tahu). Vrať strukturovaný výsledek. **Hloubka.** Co patří do modelu (pravidla) vs service (orchestrace). „Model nevolá model.“ Transakce kolem akce. **Rails / .NET.** PORO service + `ServiceResult` / **MediatR** command handler. **Reference.** <https://martinfowler.com/eaCatalog/serviceLayer.html> · <https://github.com/jbogard/MediatR> **Ověření.** Co dělá service a co model? Proč ne všechno do modelu?

Fáze 5 — Konkurence a zamykání — cca týden — TADY junior → mid

Koncept. Race conditions. Dva hráči naráz. Idempotence. **Postav.** Obal tah zámek. Přidej `idempotency_key` (dvojklik/retry nesmí zahrát kartu dvakrát). **Hloubka.** Optimistic (`lock_version` + retry na `StaleObjectError`) vs pessimistic (`with_lock` / `SELECT ... FOR UPDATE`). Izolační úrovně. Jak race *otestovat* (vlákna). **Rails / .NET.** AR locking / EF Core concurrency tokens (`RowVersion`). **Reference.** <https://blog.saeloun.com/2022/03/23/rails-7-adds-lock-with/> · <https://api.rubyonrails.org/classes/ActiveRecord/Locking/Pessimistic.html> · <https://baweaver.com/writing/2026/06/05/rails-sharp-parts-lock-is-not-a-mutex/> · <https://learn.microsoft.com/ef/core/saving/concurrency> **Ověření.** Co se stane při dvojkliku? Který zámek a proč? Jak ten race otestuješ?

Fáze 6 — HTTP API + kontrakt — cca týden

Koncept. REST sémantika. Tvůj zájem o protokoly přímo tady. **Postav.** Endpointy: vytvoř/připoj/zahraj/lízni/načti stav. Statusy: 200/201/422 (data)/409 (konflikt stavu). Auth tokenem. **Hloubka.** Metody (PUT idempotentní vs POST), status kódy a proč, idempotence, kde žije token (header vs cookie), CORS. **Rails / .NET.** Action Controller / ASP.NET Core controllers + minimal API. **Reference.** <https://developer.mozilla.org/en-US/docs/Web/HTTP> · https://guides.rubyonrails.org/action_controller_overview.html · <https://guides.rubyonrails.org/security.html> **Ověření.** Nevalidní tah → 422 nebo 409? Obhaj to. Proč je GET bezpečný a POST ne?

Fáze 7 — Event-driven design — 3–5 dní

Koncept. Observer pattern, decoupling. **Postav.** Po zahrání karty event `CardPlayed` → odběratelé: ulož Move, zapiš audit, notifikuj realtime. (`GameContext` v Unity má eventy už teď — předloha.) **Hloubka.** Proč přes event a ne přímým voláním (přidání odběratele nemění service). Sync vs async. Hranice, ať z toho není spaghetti. **Rails / .NET.** `ActiveSupport::Notifications` / `EventBus` / **MediatR** notifications. **Reference.** https://guides.rubyonrails.org/active_

[support_instrumentation.html](#) · <https://martinfowler.com/articles/201701-event-driven.html> **Ověření.** Proč audit přes event a ne přímo v PlayCardService? Co tím získáš?

Fáze 8 — Realtime — cca týden

Koncept. Push „soupeř táhl”. WebSocket vs HTTP. **Postav.** Nejdřív polling (Unity se ptá každou 1–2 s). Pak refactor na WebSocket: server pushne stav všem u stolu. **Hloubka.** WS handshake (HTTP upgrade) vs request/response. Stavové spojení. Pub/sub backend přes víc procesů. **Rails / .NET.** **ActionCable** (Solid Cable bez Redisu / Redis adapter) / **SignalR** (volitelně Redis backplane). **Reference.** https://guides.rubyonrails.org/action_cable_overview.html · <https://gorails.com/series/realtime-group-chat-with-hotwire> · <https://oneuptime.com/blog/post/2026-01-29-realtime-apps-signalr-dotnet/view> · referenční tahová hra na ASP.NET Core+SignalR+EF Core: <https://github.com/phuchautea/UniXiangqi> **Ověření.** Čím se WS handshake liší od HTTP requestu? Proč single-process pub/sub v produkci nestačí?

Fáze 9 — Docker + CI/CD — 3–5 dní

Koncept. Reprodukovatelné prostředí. Zelený build po pushi. **Postav.** `compose.yaml`: Rails + Postgres jedním příkazem. GitHub Actions: `install` → `testy` → `rubocop`. **Hloubka.** Multi-stage Dockerfile (build vs runtime), non-root, `host: db` ne `localhost`, named volume. Service containers v CI. **Reference.** <https://docs.docker.com/guides/ruby/containerize/> · <https://danielabaron.me/blog/rails-postgres-docker/> · <https://github.com/actions/starter-workflows/blob/main/ci/rubyonrails.yml> **Ověření.** Spustí nový člověk projekt za 5 minut z čistého stroje? Proč multi-stage build?

Fáze 10 — Bonus (vyber 1) —

- **Replay hry** — těžší ze seed + Moves (pokud jsi šel cestou B v Fázi 2).
- **Leaderboard** — **Redis sorted set** (ZADD/ZREVRANGE). <https://redis.io/docs/latest/develop/data-types/sorted-sets/>
- **AI hráč** — server-side bot (logiku máš v Unity `GameState/CardManager`).

Joby: Solid Queue vs Sidekiq vs Redis

Nejsou to konkurenti z jedné police:

Funkce	DB-based (bez Redisu)	Redis-based
Background joby	Solid Queue (default Rails 8)	Sidekiq
Cache	Solid Cache	Redis cache
ActionCable pub/sub	Solid Cable	redis adapter

Redis = úložiště/datové struktury, ne job systém. Přidej ho až pro konkrétní důvod (leaderboard sorted set, hot cache). Pro tenhle projekt **Solid Queue stačí, Redis nepotřebuješ**. Detail: Solid Queue polluje DB přes `FOR UPDATE SKIP LOCKED` a umí enqueue ve stejné transakci jako data.

Průběžně (10 min/den) — Fowler

<https://martinfowler.com/articles/201701-event-driven.html> (Event-Driven) · <https://martinfowler.com/eaacatalog/domainModel.html> (Domain Model) · <https://martinfowler.com/bliki/CQRS.html> (CQRS — jen pochopit)

Co přeskočit (příští 2 měsíce)

Kubernetes · Kafka · Microservices · React · GraphQL

Výsledek

multiplayer backend · state machine · observer pattern · testy · concurrence/locking · HTTP kontrakt · realtime · Docker · CI/CD · portfolio projekt · základ pro JINGI